

Overlapping MoE Synchronization and Data Transfer with In-Network Computing

主线：在具有入口与尾部协调的 EP 路径上，将 READY 汇总与数据上传重叠，并在目标数据输出完成后生成有序 DONE，减少控制等待在关键路径上的暴露。

1. Introduction — 引言

- **问题：**小 batch 缩短了 payload 阶段，但跨 rank 协调的成本不会等比例下降。
- **研究场景：**DeepEP V2 跨机 Direct Dispatch/Combine，具有 pre-barrier 和发送收尾之后的全 EP 完成信号交换。
- **方法：**rank 本地就绪后上传至 INC 暂存；INC 汇总 READY 后放行，并在逐目标尾包后追加 DONE。
- **贡献：**基线同步测量、网内协调设计及其时延分析。引言用一个真实推理结果说明动机，完整数据放在 §5；当前没有 INC 原型加速测量。

2. Background and Motivation — 背景与动机

2.1 Expert-Parallel Communication

说明 Dispatch → expert compute → Combine；定义 token、rank、topk 和路由。路由决定各目标的数据量，通信 staging 提供落点，最终 expert 布局可依赖 count 交换。

rank 通过集合操作或点对点写入与轮询，交换路由、计数及就绪/完成信息。典型组织方式包括：NCCL EP HT 先汇总路由再传数据；DeepEP V2 Direct 先做就绪同步，再将计数处理与数据传输并行；LL 利用预留槽位和细粒度信号，无需独立 pre-barrier。这些路径都需保证接收资源可写、数据对消费者可见及 buffer 安全复用。

2.2 Synchronization in DeepEP V2 Direct

跨机 Direct 的路径：pre-barrier → DATA 与元数据处理 → 发送收尾及本地汇合 → 全 EP post-barrier 信号交换。本文聚焦这条路径，保留资源可写、数据可见和 buffer 安全复用的要求。

固定 EP 规模时，token 减少会缩短 payload，却不减少参与协调的 rank。EP8 pre-barrier 约 15 μ s，占比从 $T/\text{rank}=8$ 时的约 20% 降至 $T=128$ 时的约 5%，说明小 batch 下有必要减少暴露的同步开销。

2.3 Opportunities for In-Network Computing

INC 背景：网内计算支持数据路径上的状态维护与聚合，已用于集合归约和 MoE 通信。本文进一步利用有容量保障的暂存与保序通知能力处理同步。

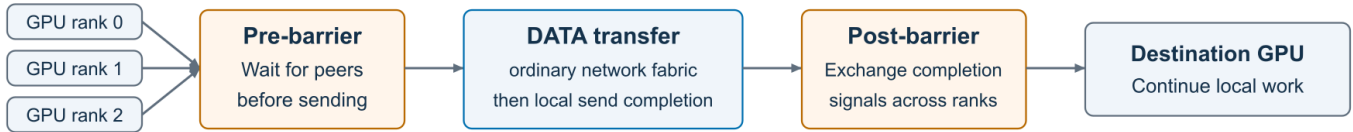
优化机会：本地就绪后先上传，INC 同时汇总 READY；目标可写且所需条件满足后放行。INC 跟踪目标输出，在尾数据后追加有序完成通知，保证通知可见时数据已可见；真正的全局依赖仍保留。

Key Insight：同步条件必须满足，但同步通信可以与数据传输重叠：上传时汇总就绪，尾数据携带完成通知，从而减少关键路径上的同步等待。

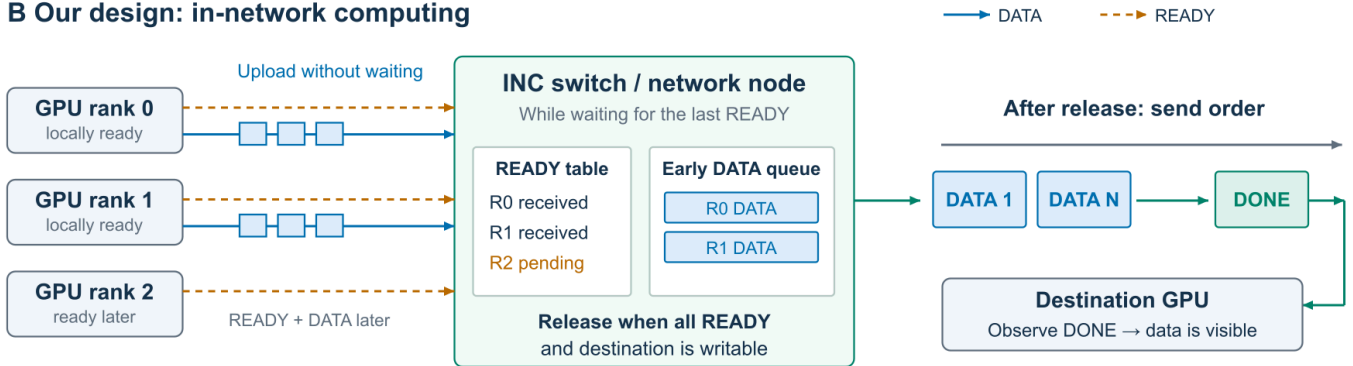
3. Design — 设计

3.1 Overview and Assumptions

A Direct baseline



B Our design: in-network computing



1 Upload while collecting READY

2 Append DONE after final DATA

INC 记录哪些 rank 已就绪、各目标预期接收和已转发的 token / 归约结果数量，以及各接收 buffer 用于哪一轮。INC 同时缓存早到数据。generation 标识一次 Dispatch 或 Combine，epoch 区分同一 buffer 的不同轮使用。所有相关数据必须经过 INC；多级转发或多 rail 路径需要确认每一段都已完成。

接收 buffer 在初始化时按 symmetric memory 分配、注册，布局、地址映射和访问权限一次性提供给 INC。token 的路由和标识确定目标 buffer 及写入偏移，不依赖全局计数。READY 确认上一轮已用完接收 buffer、本轮可以写入。发送端直接上传，INC 缓存压力由 PFC 回压控制。

3.2 Readiness Aggregation and Early Upload

rank 完成本地路由、计数和接收区复用检查后，将 READY、epoch 与逐目标 count vector 一起发送，随后直接上传 DATA，不申请暂存额度。INC 收到本轮完整消息才标记其就绪，并从第一份向量到达起流式累加；重复或过期消息不更新状态。所需 READY 收齐且目标 staging 可写后立即按路由 fan-out，不必等待向量求和结束。

全局接收计数和 expert buffer 中的最终位置可与数据传输并行计算。硬件加法可流水执行，未完成的求和仍会延迟 DONE。预期计数和转发计数都按各目标实际接收的 token 副本计数。

3.3 Completion Notification

INC 对所有 source 的 count vector 求和，得到各目标应接收的 token 数，并在完整 token payload 提交发送后更新转发计数，每份 token 副本只计一次。DONE 的条件是：(i) 所有向量求和完成，(ii) 目标已就绪，(iii) 预期 token payload 已全部提交发送。保证：

目标观察到 DONE ⇒ 本轮覆盖的全部数据已对目标 GPU 可见。

DONE 表示该目标的输入数据已到齐；开始计算前仍需满足其他依赖。

Combine 根据 Dispatch 中记录的 expert 分配，确定每个 token 应收到哪些 expert outputs。输出携带 top-k slot，即使 expert 位于同一 rank 也能区分。INC 对加权输出求和，所有预期输出到齐且目标的 reduced results 全部提交发送后，按相同保序要求发送 DONE。

3.4 Buffer Management and Progress

收到 DONE 并满足其他依赖后，Dispatch 将 token 按 expert 排列以便计算；Combine 将归约结果交给后续计算。最后一次读取结束后才能复用接收 buffer，下一轮 READY 确认这一点。发送 buffer 则需等传输层读取完成后才能复用。

INC 暂存占用触发 PFC，在溢出前暂停上游 DATA，并为暂停生效前的在途数据预留 headroom；空间恢复后沿原路径继续，无需每轮 credit 交换。READY/count vector 使用独立控制优先级和预留资源，避免被 DATA 暂停而无法放行。归约状态等资源限制也需联动回压；压力下重叠和吞吐可能下降。

4. Latency Analysis — 时延分析

4.1 Delay Model

借鉴 Swift 区分端点与网络时延的思路：消息时延 = 发送端发起时间 + 网络传输时间 + 接收端观察时间。网络项包含传播、序列化和排队；barrier 还包含各 rank 就绪时间不同造成的等待及本地同步，因此其时长不能直接当作网络时延或 INC 收益。

比较基线与 INC 何时完成数据交付和通知，使目标具备开始后续计算的条件。无需引入 Swift 原始的六个时间戳。

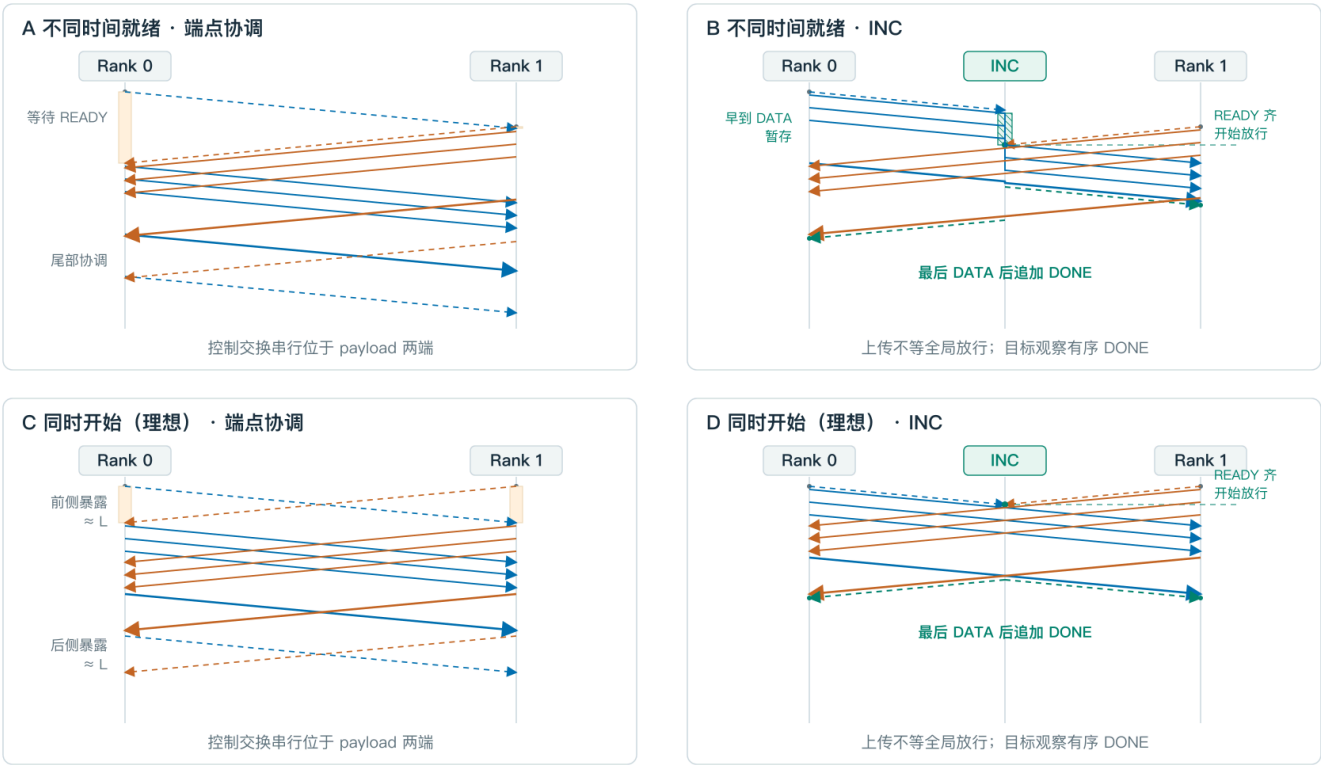
4.2 Overlapping the Pre-Barrier

先讨论各 rank 不同时就绪的一般情况：基线等自己与 peer 就绪后才发送，INC 允许本地就绪的源直接上传，目标写入仍等 READY 和目标资源到齐。PFC 暂停会延迟数据到达 INC；暂存、慢 rank 和出口服务共同决定完成时间，不能将提前上传量直接换算为加速。

INC 的两项收益：提前上传，随尾包通知完成

上排：一般到达情况 下排：理想参考 颜色区分两个发送方向，线型区分数据与控制

—— 实线：DATA - - - - - 虚线：READY / 完成 signal - - - - - 绿色尾标：DONE 斜线区：暂存等待



理想参考：L = RTT_fabric / 2；前后各约减少 L，另计 INC 开销。一般情况由就绪偏斜、尾部暴露与出口服务共同决定。

4.3 Overlapping the Post-Barrier

区分发送侧收尾、目标数据可见、完成通知被观察三个时刻，比较同一目标的通信完成时间：

尾部净收益 = 基线通信完成时间 - INC 通信完成时间。

窄 signal 窗口未直接测得数据可见后仍需等待多久。理想情况下，各 rank 同时就绪，前后各暴露一次单向控制传播，INC 转发速率不变且计数及时完成，则收益为 $2L - \epsilon$ ：一个网络 RTT 减去新增开销。一般情况还需考虑数据到达、排队、计数与通知时间，不能直接相加实测的前后同步时长。

5. Evaluation — 实验评估

本章测量基线的同步开销及其在推理中的影响；INC 的端到端收益仍需实现后评估。

5.1 Experimental Setup and Methodology

- H200：两台各八卡，机内 NVSwitch，机间八轨 400GbE RoCE；跨机使用 GDAKI3/TC162，DeepEP V2 Direct。
- 微基准：H=7168、topk=8、E=256、BF16、balanced/uniform。每档三次独立进程、每次 200 次有效迭代；post-barrier T=32 合并两批共六轮。
- 每次迭代取该阶段耗时最短的 rank；占比除以该 rank 同次 trace-on 算子时长。均值误差表示轮间标准差，不把不同 rank 的阶段占比相加。
- 总算子时延来自 trace-off，阶段来自 trace-on。推理的 GPU-step 占比使用同 rank/step 配对并跨 rank 平均，与微基准 rank-min 统计不同。
- 实测使用通信插桩；推理还含 expert-padding 正确性修复。所有数值均为已有软件基线观测。

5.2 Pre-Barrier

主图同时展示 pre-barrier 绝对时长和算子占比。EP8 的 pre-barrier 维持约 15 μ s，占比随 T 增大而下降。

T/rank	Dispatch pre-barrier 占比	Combine pre-barrier 占比
8	(20.57 $\pm$ 0.06)%	(19.84 $\pm$ 0.13)%
32	(12.25 $\pm$ 0.17)%	(12.26 $\pm$ 0.26)%
64	(8.29 $\pm$ 0.51)%	(8.20 $\pm$ 0.16)%
128	(4.88 $\pm$ 0.04)%	(5.05 $\pm$ 0.19)%

各固定 EP 配置下对可用 T 档作常数描述；所有列出的点参与估计，不再称为独立预测验证。

EP 卡数	参与估计的 T/rank	Dispatch (μ s)	Combine (μ s)	各档均值最大偏差 (μ s)
4	8/32/96/128	11.65	11.54	0.06
8	8/32/64/96/128	14.96	14.94	0.16
16	8/32/96/128	18.36	18.28	0.09

EP8 的 T=64 是后续不同插桩批次补测。不同 EP 配置也改变每机卡数与网络并发，不据此外推任意 rank 数的规律。
来源：配对数值、pre-barrier 常数记录；主图源码。

5.3 Post-Barrier

此处 post-barrier 数据仅指信号交换窗口：从本地发起信号到收齐全部预期信号，排除此前的 flush 和本地汇合。保留轮间波动，不再展示宽 post 拟合。

T/rank	Dispatch post-barrier (μs)	Combine post-barrier (μs)
8	13.97 ± 0.04	13.56 ± 0.24
16	13.98 ± 0.23	13.78 ± 0.18
32	11.67 ± 0.94	11.70 ± 1.43
64	11.81 ± 0.42	11.76 ± 0.29
96	11.12 ± 0.22	11.24 ± 0.92
128	11.73 ± 0.07	11.88 ± 0.18

T=8/16 约为 14 μs，已测 T≥32 约为 11–12 μs；T=32 的慢轮次全部保留。固定 EP8 时信号数量不随 T 改变，但窗口仍可能受 peer 就绪差、端点和网络排队影响。尚无证据支持协议阈值或通用时延公式，也不能将窗口全部计为 INC 收益。来源：[复测与代码分析](#)；[主图源码](#)。

5.4 Synchronization in MoE Inference

Qwen3–30B–A3B, BF16, EP8 Direct, decode 输入/输出各 128 token。比较同一 EP 规模和并发下的单机与跨机 pre-barrier 占比。

拓扑	并发	实测 token/rank	D+C pre-barrier 占 GPU step
单机 1n8	64	8	9.33%
跨机 2n4	64	8	16.72%
单机 1n8	128	16	7.94%
跨机 2n4	128	16	15.19%

跨机约 15%–17%，单机约 8%–9%。这是采样 GPU step 的占比，不是请求延迟或 INC 加速上限；三次 client 窗口复用服务实例，未测真实模型中 post-barrier signal 窗口的逐层占比。来源：[端到端数值](#)、[pre-barrier 分项](#)；[主图源码](#)。

6. Related Work — 相关工作

6.1 EP Communication

[DeepEP](#)、[NCCL EP](#) 提供基线及不同同步组织；[SwiftEP](#)、[UEP](#)、[Perseus](#)、[UBEP](#) 分别从传输、可移植接口或流水内同步等方向优化 EP。比较其处理的依赖与本文的网络 READY 汇总、暂存放行、目标完成通知，避免反复逐篇否定。

6.2 In-Network Computation

[DySHARP](#)、[MultiWrite](#) 提供 MoE multicast/reduction 与写语义；[SHARP](#)、[SwitchML](#) 提供集合卸载与归约状态管理先例。本文借用这些数据面能力，贡献集中在协调时序。

6.3 Synchronization Offload

[EPIC](#) 提供 INC 协议与资源抽象；[GPU-Initiated Networking](#) 提供 PUT/signal/fence 等端点原语。比较控制状态放置位置及完成语义。Swift 和测量工具在相应方法处引用，不另设一组重复介绍。

7. Conclusion — 结论

跨机 Direct 的同步成本在小 batch 下占比显著。本文提出通过网内就绪汇总和有序完成通知重叠两端协调，并给出所需的可见性、资源与进展条件。现有实验支撑问题动机，实际收益仍需 INC 实现验证。